

Soaring Research — ArduPilot vs iNav (and what UAVX ArmQ should adopt)

Research into how the two mainstream fixed-wing open-source autopilots implement soaring / thermalling, cross-referenced against the (dead-gated) ArduSoar-derived code already sitting in UAVXArmQ/src/soar/. Sources are the current upstream source trees, not marketing docs:

- ArduPilot libraries/AP_Soaring/* (AP_Soaring.cpp/.h, Variometer.cpp, Extended-KalmanFilter.cpp, SpeedToFly.cpp) and ArduPlane/mode_thermal.cpp.
- iNav src/main/navigation/navigation.h + navigation_fixedwing.c.

Companion doc: Soaring_Status_And_Development.md (UAVX state machine + dead-gate analysis).

1. ArduPilot — full autonomous soaring

ArduPilot is the reference implementation for real cross-country thermal soaring. libraries/AP_Soaring/ implements the entire loop.

1.1 Parameter surface (all tuning is parameters, not constants)

Param	Default	Meaning
SOAR_ENABLE	0/1	master enable
SOAR_VSPEED	0.7 m/s	netto climb needed to enter a thermal
SOAR_Q1	0.001	EKF process noise: thermal strength
SOAR_Q2	0.03	EKF process noise: position + radius
SOAR_R	0.45	EKF measurement noise (vario)
SOAR_DIST_AHEAD	5 m	initial thermal-centre guess ahead of nose
SOAR_MIN_THML_S	20 s	min time committed to a thermal
SOAR_MIN_CRSE_S	10 s	min cruise time before re-thermalling
SOAR_POLAR_CD0/B/K	0.027 / 0.031 / 25.6	drag polar: wing efficiency
SOAR_ALT_MAX	350 m	don't thermal above this (rel home)
SOAR_ALT_MIN	50 m	don't glide below this
SOAR_ALT_CUTOFF	250 m	cut throttle at this alt on climb
SOAR_MAX_DRIFT	-1 (off)	abort thermal if drifted past this
SOAR_MAX_RADIUS	-1 (off)	RTL if thermal-in-progress beyond this
SOAR_THML_BANK	30°	circle bank angle
SOAR_THML_ARSPD / SOAR_CRSE_ARSPD	0	thermal / cruise airspeed override
SOAR_THML_FLAP	0	flap % while circling

Everything is a param — nothing is a `#define`. This is the direct thing UAVX is missing (our `soar.h` is 100% `#define`).

1.2 Architecture — clean 4-module split

1. **Variometer** — the measurement layer. Computes the *netto* (airmass) vertical speed:

```
reading = raw_climb_rate           (EKF vertical velocity)
          + dsp_cor * aspd / g      (total-energy term from accel bias-
corrected)
          + sinkrate(roll)          (polar: CD0/B, mclaurin cos(phi))
```

Filters every quantity over a **circling time constant** $\tau = 2 \cdot \pi \cdot V / (g \cdot \tan(\text{bank}))$ — e.g. one full orbit, so rotating around the thermal doesn't confuse the climb measurement. `_expected_thermallling_sink` is the polar sink rate *while turning*, used to compare “how much better is this thermal than just circling”.

2. **ExtendedKalmanFilter** — 4-state {strength, radius, position.X/Y} fitting a Gaussian updraft, exactly the CSz/Tabor model UAVX already ported (`soar/ekf.c`). Measurement = vario. Jacobian computed analytically from the Gaussian. Radius clamped ≥ 40 . Q from process noise; P forced symmetric.
3. **SpeedToFly** — MacCready-style optimal *cruise* airspeed between thermals, solved from the polar + measured headwind + expected next-thermal lift + sink. Mostly optional (only used when `SOAR_CRSE_ARSPD < 0`).
4. **SoaringController** — the state machine tying it together: `suppress_throttle()` (glide while above cutoff), the enter/exit criteria, thermostat alt band, hysteresis timers, drift check.

1.3 Mode integration (ArduPlane/mode_thermal.cpp)

Soaring does **not** live inside the nav controller. It is a separate *flight mode* (THERMAL, RC-selectable or auto-commanded) that:

- **_enter()**: only if soaring active; does `do_loiter_at_location()` + `init_thermallling()`; target = thermal centre ahead of nose.
- **loiter radius** from `get_thermallling_radius() = V^2 / (g \cdot \tan(\text{THML\_BANK}))`.
- Runs the thermal EKF at **fixed 50 Hz**, evaluating:
 - keep circling if `check_cruise_criteria() == GOOD_TO_KEEP_LOITERING`;
 - exit when summed altitude drift / thermal too weak (`SOAR_MIN_THML_S` vs McCready), out of alt band, `MAX_DRIFT`, `MAX_RADIUS` → **RTL**, or RC exit request;
 - heading-aligned exit (don't flit off mid-turn without being lined up for the next WP / home / cruise heading).

1.4 Key design strengths

- **Every tuning knob is a runtime parameter** — traceable, tunable, GCS visible.
- **Altitude band as a state machine**, not just a clamp: `ALT_MAX` kills thermalling, `ALT_MIN` forces motor back on, `ALT_CUTOFF` kills throttle on the powered climb — giving the “climb hard then glide” cruise.
- **Explicit hysteresis** (`MIN_THML_S / MIN_CRSE_S`) prevents thermal↔glide thrash.
- **The thermal is a positioned object** (EKF), so the plane reacquires the same lift after exiting — it's not blind hand-off.
- **Speed-to-fly** optimizes the cruise, not just the climb.

2. iNav — bare minimum: motor-off in a loiter

iNav soaring is **deliberately minimal** — it contributes almost nothing over “turn the motor off and let the pilot (or existing loiter nav) handle it”:

2.1 Config surface

Two items only:

- NAV flight-mode SOARING_MODE (FLIGHT_MODE(SOARING_MODE)).
- Config bit soaring_motor_stop (navigation.h, in the general.flags bitfield: uint8_t soaring_motor_stop; — stop motor when Soaring mode enabled). Optional fw.soaring_pitch_deadband.

2.2 Behavior — everything in navigation_fixedwing.c

```
if (getMotorStatus() == MOTOR_STOPPED_USER || FLIGHT_MODE(SOARING_MODE)) {  
    // Motor has been stopped by user or soaring mode enabled to override altitude control  
    resetFixedWingAltitudeController();  
    setDesiredPosition(&...pos, yaw, NAV_POS_UPDATE_Z);  
}  
and  
if (FLIGHT_MODE(SOARING_MODE) && navConfig()->general.flags.soaring_motor_stop) {  
    ENABLE_STATE(NAV_MOTOR_STOP_OR_IDLE);  
}
```

What that actually does:

1. **Motor stop / idle** (MOTOR_STOP_OR_IDLE) when soaring.
2. **Alt-hold released** — altitude controller reset + desired position re-anchored at the current spot. The plane then flies the exact same glide task as a pure coast; nothing commands it to circle a tighter circle.
3. **What is BY DESIGN** — iNav leaves thermalling to the pilot steering the plane. The pilot circles, the autopilot holds heading. No thermal EKF, no thermal-centring, no vario, no netto, no MacCready, no speed-to-fly, no auto mode-switching, no hysteresis.

So iNav soaring is compatible with slope/Duration-style glide soaring — the plane coasts and the pilot (or loiter) holds the circle. It is NOT a thermal autopilot at all.

3. Comparison matrix

Capability	ArduPilot	iNav	UAVX ArmQ (soar/)
Total-energy / netto vario	[ok] merged EKF vel + TE + polar sink	[x]	[!] USE_NETTO stub, off
Thermal-centre EKF	[ok] 4-state Gaussian	[x]	[ok] present (ArduSoar clone)
Auto enter/exit thermal	[ok] mode-switch + hysteresis	[x]	[!] state machine present, dead-gated
Throttle-off glide	[ok] sup-press_throttle()	[ok] motor-stop	[ok] ThrottleSup-pressed, dead
Airspeed / speed-to-fly	[ok] SpeedToFly + airspeed	[x]	[!] constant

Capability	ArduPilot	iNav	UAVX ArmQ (soar/)
Drift/RTL guard	[ok] MAX_DRIFT/MAX_RADIUS	[x]	[x]
Altitude band clamp	[ok] ALT_MIN/MAX/CUTOFF	[x]	[!] InAltitudeBand yes, no cutoff logic
Tunable params	[ok] all SOAR_* params	[!] config bits only	[x] all #define
Real-time pilot override	[ok] THERMAL mode + RC	[ok] SOARING mode	[x] none wired

iNav is not a reference for full soaring — compare UAVX against ArduPilot if the goal is autonomous cross-country thermal autopilot. iNav only confirms the minimal “motor-stop glide” model.

4. What the “experts” do — viz. for UAVX ArmQ

Ordered by what our ArduSoar-derived code actually needs (see `Soaring_Status_And_Development.md` Phases 0–2):

P0 — get the shared algorithm actually running

Do not copy a new architecture. Both mainstream implementations validate the algorithm already present in soar/ (Gaussian updraft EKF, throttle-off glide above cutoff). The only missing step is the enable gate — UAVX has the algorithm but never turns it on: - make `F.Glide` reachable (config-bit gated to `CAT_FW`), wire `SuppressThrottle()`, verify the FW throttle output path honours `ThrottleSuppressed` (not just `mixer.c:199`).

P1 — borrow the always-on wind correction ArduSoar uses

ArduSoar publishes the EKF drift **from the AHRS wind estimate every update** (`wind_drift = wind*dt*climb/strength`). Our `soar.c:157-166` blocks this — uncomment to stop the circletrack drifting cross-wind while circling.

P2 — parameterize the #defines (ArduPilot model, iNav not usable here)

The single biggest research takeaway: **ArduPilot exposes every knob as a `SOAR_* param`; ours are all `#define`**. Before tuning we must promote: `ThermalMin`, `CRUISE_MIN_MS`, `THERMAL_MIN_MS`, `AltMinM/MaxM/CutoffM`, `THERMAL_DIST_AHEAD_M`, and the EKF noise `THERMAL_Q1/Q2/R` into the unified float param table (`AGENTS.md`), mirroring the `(lo,hi)` into `GCS PARAM_LIMITS`. That’s a precondition, not a nicety.

P3 — then optionally adopt ArduPilot’s smarter strategy pieces

- **Altitude band state machine** (climb to `ALT_CUTOFF`, cut throttle, glide, throttle back at `ALT_MIN`) — exact pattern the `AGENTS.md` recap already describes for `BoostClimb`.
- **Drift / last-point guard** (`MAX_DRIFT` toward an exit, `MAX_RADIUS` RTL) for safety/site awareness.
- **SpeedToFly** optimal cruise airspeed once the base loop converges.
- **Hysteresis** (`MIN_THML_S/MIN_CRSE_S`) already in our constants, keep.

Explicitly NOT worth adopting from iNav

- **motor-stop-only model:** it's a pilot-driven passive glide, not an autopilot. The one thing worth copying is the **motor-stop (MOTOR_STOP_OR_IDLE) + suspending the altitude controller** when entering soaring anyway — but ArduPilot's `suppress_throttle()` already does the throttle side and the alt-hold keeps the plane's geometry.

iNav does confirm two aviation facts for free

1. A real **fixed-wing altitude controller must be suspended/decoupled when gliding**, else pitch output fights the climb/glide — ArduPilot (reset alt controller, anchor pos at current) does the same, just inside THERMAL.
 2. Motor-cut must be explicit & visible — iNav has a dedicated config bit + a `NAV_MOTOR_STOP_OR_IDLE` state flag; UAVX's bare `ThrottleSuppressed` boolean has no arm override / state signal.
-

5. Sources

- ArduPilot ardupilot master: `libraries/AP_Soaring/{AP_Soaring.cpp,AP_Soaring.h,SoaringController,ArduPlane/mode_thermal.cpp}`.
- iNav inav master: `src/software/navigation/navigation.h` (config flags, incl. `soaring_motor_stop`, `soaring_pitch_deadband`), `src/software/navigation/navigation_fixedwing.c`.

Notes

- Research only — no code changed. The reference for any future UAVX redesign stays ArduPilot.
- PDF conversion (`scripts/md2pdf.sh`) is the user's job; assistant does not generate PDFs.